

Project1 Report

12312515 王洛源

Q1

本小问全程在服务器上进行，相关代码文件及图片结果保存在压缩包Q1内。由于服务器与本地路径不同，直接运行代码可能会报错，如需复现需要调整代码读取文件部分。

(1)

本项目中，我选择在课程提供的 Jupyter 服务器上部署 OpenClaw，而不是安装在个人电脑本地。这样做主要有两个原因：第一，课程服务器已经提供了基于 Linux 的开发环境，并预装了 Python 和 JupyterLab，便于后续直接开展数据分析任务；第二，将 OpenClaw 部署在服务器端可以将本次项目的工作目录与个人电脑环境分离，更适合在受控环境下处理课程数据。

在安装 OpenClaw 之前，我首先对服务器环境进行了检查，包括操作系统、Python/Jupyter 是否可用，以及 Node.js 是否已经安装。检查结果表明，该服务器为 Ubuntu 环境，Python 与 JupyterLab 已经配置完成，但最初并未安装 Node.js 和 npm。因此，我先补充安装了 OpenClaw 所需的运行环境，包括 Node.js 与 npm，并在 `/data/project1_env` 下建立了独立的项目目录，用于存放配置文件、日志、临时文件和后续分析工作区。在完成这些准备后，我成功安装了 OpenClaw，并完成了初步的 provider 配置。

1. 整体架构

OpenClaw 可以理解为一个将大语言模型与本地执行环境连接起来的智能代理框架，其整体上可以分为以下四个部分：

1. 用户交互层：负责接收用户输入的任务指令，通常通过终端或 TUI 界面进行交互。
2. 模型提供层 (LLM provider)：负责连接外部大语言模型服务。在本项目中，后续配置为我自行购买的千问相关 API。
3. 技能/工具层 (skills/tools)：负责执行具体任务，例如代码生成、命令执行、文件操作和数据处理等。
4. 本地执行环境：即课程提供的 Linux/Jupyter 服务器，用于实际执行命令、读取文件并生成分析结果。

2. 工作机制

OpenClaw 的基本工作流程如下：

- 用户输入任务要求；
- OpenClaw 将任务发送给已经配置好的大语言模型；
- 模型根据任务内容决定是直接回答，还是调用相应的技能/工具；
- 被调用的工具在本地环境中执行；
- 执行结果再返回给模型，并最终呈现给用户。

因此，OpenClaw 不只是一个像平常我们使用的deepseek，豆包等LLM对话系统，更像是一个能够与本地分析环境协同工作的任务执行代理。

3. 与数据分析相关的三个能力

在本次项目中，OpenClaw 中与数据分析最相关的能力主要包括以下三类：

1. 文件与工作区交互能力

它可以读取文件、检查目录结构并组织输出结果，这对于管理数据集、分析脚本和结果文件非常有帮助。

2. 命令执行能力

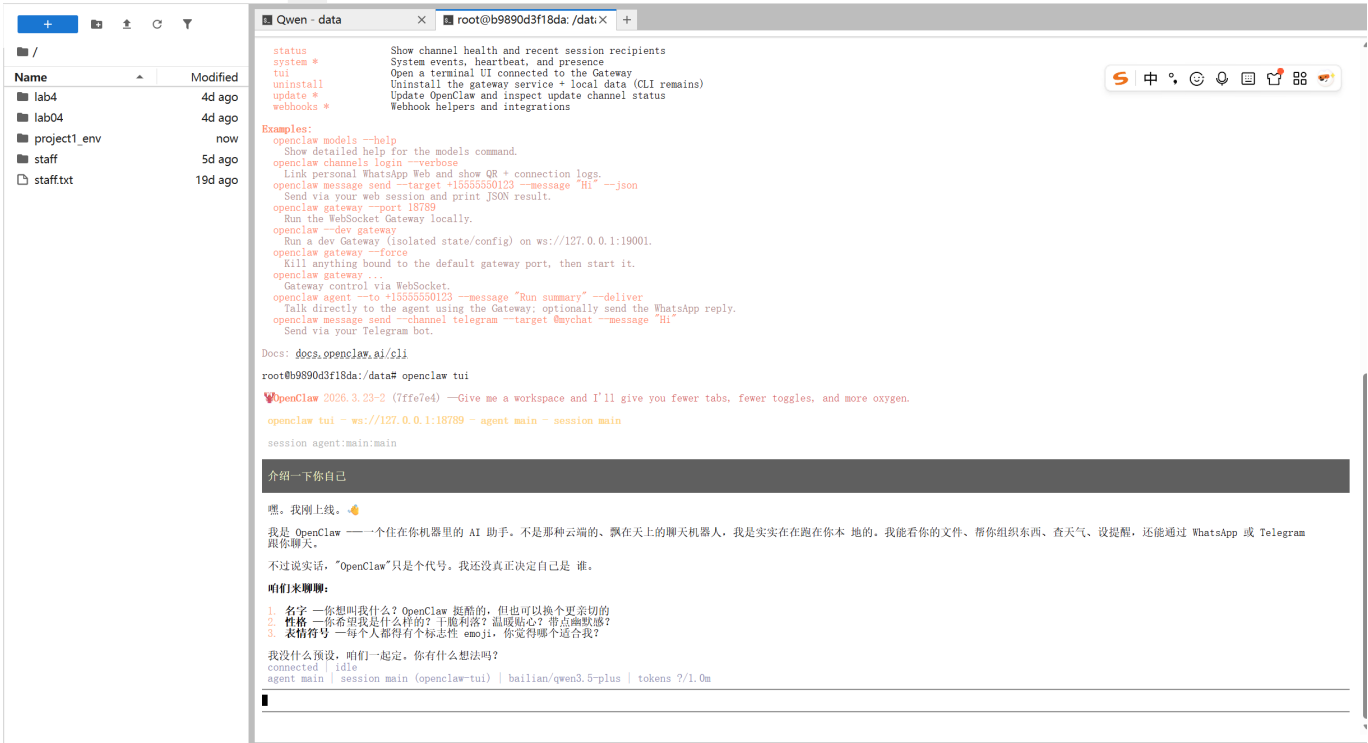
它可以辅助生成并执行 shell 命令或分析相关命令，用于检查依赖、运行程序和管理分析流程。

3. 数据分析代码辅助能力

它可以帮助生成或修改数据分析代码，例如数据读取、描述性统计和结果整理等，这与本项目后续基于 Spark 的分析任务直接相关。

总体而言，OpenClaw 提供了一种将大语言模型与本地数据分析环境结合起来的工作方式，在服务器环境中尤其适合用于组织和辅助完成数据分析任务。

下面是我在服务器上成功配置openclaw，并与之进行第一次对话的截图



(2)

首先我上传了数据文件到服务器并且解压，然后将这一小问的任务进行拆分，交给openclaw完成。下面是我们的对话：

可以，后续任务都在这个目录下完成，除非我特别说明否则别跑到你那边去了。现在请你生成代码文件，任务如下：用 PySpark 读取文件 diabetes_012_health_indicators_BRFSS2015.csv，创建 Spark DataFrame，检查并展示 schema

搞定。

代码文件： /data/project1/read_data.py

运行结果：

- 成功读取 CSV 并创建 DataFrame
- 数据量： 253,680 行， 22 列
- Schema： 所有字段都是 double 类型（数值型）

主要字段：

- Diabetes_012 - 糖尿病指标（目标变量）
- HighBP, HighChol - 高血压、高胆固醇
- BMI - 身体质量指数
- Smoker, Stroke, HeartDiseaseorAttack - 吸烟、中风、心脏病
- PhysActivity, Fruits, Veggies - 运动、水果蔬菜摄入
- GenHlth, MentHlth, PhysHlth - 整体健康、心理健康、身体健康评分
- Age, Sex, Education, Income - 人口统计信息

代码就在 /data/project1/ 目录下，后续任务我会继续在这里完成。

connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens ?/1.0m

好，下一个代码任务是基于已经读入的 Spark DataFrame，完成下列任务：统计 Diabetes_012 每个类别的样本数，计算每个类别的占比。要求分别给出 DataFrame API 和 Spark SQL API 两种写法，保存为两个不同的代码文件，输出结果表按 Diabetes_012 升序排列

完成。两个代码文件都已创建并运行：

文件列表：

- /data/project1/stat_dataframe_api.py - DataFrame API 写法
- /data/project1/stat_sql_api.py - Spark SQL API 写法
- 输出结果分别保存在 stat_dataframe_api_output.txt 和 stat_sql_api_output.txt

统计结果（两种方法结果一致）：

Diabetes_012	类别	样本数	占比
0.0	无糖尿病	213,703	84.24%
1.0	有糖尿病	4,631	1.83%
2.0	有糖尿病	35,346	13.93%

总计： 253,680 样本

数据分布明显不均衡，无糖尿病样本占绝大多数（84.24%）。

connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens ?/1.0m

接下来在上面的基础上，编写代码完成这个任务：请基于 Diabetes_012 分组，对以下五个变量做描述性分析：HighBP, HighChol, BMI, PhysActivity, Age。要求还是分别给出 DataFrame API 和 Spark SQL API 两种写法，对二元变量（HighBP, HighChol, PhysActivity）统计各组中取值为1的比例，对连续或有序变量（BMI, Age）统计各组的均值、标准差、最小值、最大值，然后输出结果表，最后写一段简要比较，说明不同 Diabetes_012 类别之间的差异

完成。两个代码文件已创建并运行：

文件列表：
- /data/project1/descriptive_dataframe_api.py - DataFrame API 写法
- /data/project1/descriptive_sql_api.py - Spark SQL API 写法
- 输出结果分别保存在对应的 _output.txt 文件中

运行结果汇总

【二元变量】各组中取值为 1 的比例 (%)

Diabetes_012	类别	HighBP (高血压)	HighChol (高胆固醇)	PhysActivity (运动)
0.0	无糖尿病	47.53%	36.95%	77.91%
1.0	糖尿病 (1 级)	62.90%	62.08%	67.85%
2.0	糖尿病 (2 级)	75.27%	67.01%	63.05%

【连续变量】描述性统计

Diabetes_012	BMI 均值	BMI 标准差	BMI 范围	Age 均值	Age 标准差	Age 范围
0.0	27.74	6.26	[12.0, 98.0]	7.79	3.10	[1.0, 13.0]
1.0	30.72	6.96	[13.0, 96.0]	9.08	2.63	[1.0, 13.0]
2.0	31.94	7.36	[13.0, 98.0]	9.38	2.33	[1.0, 13.0]

比较分析

- 高血压**：糖尿病组的高血压比例显著更高（62.9%~75.3% vs 47.5%），且随糖尿病严重程度递增
- 高胆固醇**：糖尿病组比例更高（62%~67% vs 37%），差异明显
- 运动习惯**：无糖尿病组运动比例最高（77.9%），糖尿病组较低（63%~68%）
- BMI**：糖尿病组平均 BMI 更高（30.7~31.9 vs 27.7），肥胖与糖尿病强相关
- 年龄**：糖尿病组平均年龄更高，年龄增长是风险因素

结论：糖尿病患者有多项健康指标上均表现更差，符合医学认知。

connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens ?/1.0m

根据你前面的分析，总结一下哪些因素看起来与 diabetes risk 最相关

根据前面的数据，按相关性强度排序：

与糖尿病风险最相关的因素

强相关

- 高胆固醇 (HighChol)** 一差异最大
 - 无糖尿病组：36.95%
 - 糖尿病组：62%~67%
 - 差距约 25-30 个百分点，代谢问题高度集中
- 高血压 (HighBP)** 一次强
 - 无糖尿病组：47.53%
 - 糖尿病组：63%~75%
 - 差距约 15-28 个百分点，心血管风险叠加
- BMI** 一连续变量中最明显
 - 无糖尿病组：27.74
 - 糖尿病组：30.7~31.9
 - 差距 3-4 个点，肥胖是明确风险因素

中等相关

- 运动习惯 (PhysActivity)** 一负相关
 - 无糖尿病组：77.91%
 - 糖尿病组：63%~68%
 - 差距约 10-15 个百分点，缺乏运动增加风险
- 年龄 (Age)** 一有影响但不如上述因素
 - 差距约 1.3-1.6 个点，年龄增长确实相关，但效应相对温和

总结：代谢综合征的几个核心指标（高胆固醇、高血压、肥胖）与糖尿病风险关联最强，运动是可控的保护因素。年龄有影响，但不如生活方式和代谢指标来得明显。

说白了一管住嘴、迈开腿、控制体重，比担心年龄实在得多。

connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens ?/1.0m

随后我对openclaw的代码进行了初步检查，均可以运行并且能给出预期的结果，输出文件格式也正确，总结也比较合理。

(3)

下面是openclaw总结出的几条结论：可以看得出这些结论符合我们日常认知，从数据上看也能得到支撑。

Observation	相关系数	结论
BMI 与糖尿病风险	0.224	✓正相关
年龄与糖尿病风险	0.186	✓正相关
高血压与糖尿病风险	0.270	✓正相关

关键发现

Observation 1 - BMI:

- 体重正常 (BMI 18.5-25): 糖尿病发病率 7%
- 超重 (BMI 25-30): 14%
- 肥胖 I 级 (BMI 30-40): 26%
- 肥胖 II 级 (BMI 40+): **38%**

Observation 2 - 年龄:

- 18-29 岁: 2%
- 30-39 岁: 5%
- 40-49 岁: 9%
- 50-59 岁: 15%
- 60-69 岁: 21%
- 70+ 岁: **23%**

Observation 3 - 高血压:

- 无高血压: 7%
- 有高血压: **27%** (接近 4 倍风险)

(4)

我在完成第 (3) 小问的任务时，前一天先让openclaw生成了五条可能的结论，后续我计划第二天再从中选择三条，完成后面的数据处理工作。但我第二天重新进入服务器准备让openclaw在前一天的工作基础上继续完成任务时，发现它忘记了前天的工作内容，也搞错了工作目录：

对于你刚才挑出的五条observation，选出最明显最容
代码并运行。代码需要能绘制相关图表，对于不同的o

我没有找到之前关于五条 observation 的对话记录。

你生成的文件放到哪里了？能放到/project1这个文件夹下吗？

已完成！所有文件已移动到 /project1 目录：

- /project1/validate_observations.py - PySpark 验证代码
- /project1/observation_validation.png - 三图并排对比
- /project1/bmi_distribution_boxplot.png - BMI 分布箱线图

你放错了，应该是/data/project1

于是我在先重新完成 (3) 之后，认为可能是因为数据没有保存导致的，于是我详细询问了openclaw哪些内容会被它记录，哪些不会，结果如下：

我的记忆系统是这样的：

类型	存储位置	是否持久化
对话历史	会话内存	✗重启消失
身份人格	SOUL.md， IDENTITY.md	✓持久化
长期记忆	MEMORY.md	✓持久化（但还没创建）
每日日志	memory/YYYY-MM-DD.md	✓持久化（但还没创建）

你的每日日志和长期记忆根据什么规则创建？

根据 AGENTS.md 的规定，我的记忆创建规则是这样的：

📅每日日志 (memory/YYYY-MM-DD.md)

什么时候创建：

- 每天第一次会话时自动创建
- 记录当天的“原始日志”——发生了什么、做了什么、用户说了什么重要 的话

记录什么：

- 完成的任务
- 用户的偏好和指示
- 遇到的问题或教训
- 任何值得记住的上下文

不记录什么：

- 机密信息（密码、密钥等）
- 琐碎的闲聊

🧠长期记忆 (MEMORY.md)

什么时候更新：

- 每隔几天（通常在 heartbeat 时）
- 当发生真正重要的事情时

记录什么：

- 重要的决定和原因
- 用户的长期偏好
- 学到的教训
- 项目进展的关键节点
- 人际关系相关的信息

维护规则：

- 定期回顾 memory/ 里的 daily files
- 把值得长期保存的内容提炼到 MEMORY.md
- 删除过时的信息

通过这个小插曲，我对openclaw的工作方式有了更深入的了解。

我认为openclaw在大数据分析任务中，相对于其他AI产品的明显优势在于由于openclaw有较高的本地权限，能很方便快捷的完成复杂任务并执行自我纠错。不过这也带来了数据安全等问题。

Q2

本小问同样全程在服务器上进行，相关代码文件及图片结果保存在压缩包Q2内。

(1)

整合出的ipynb文件为survival_analysis.ipynb，相关生成的图表和分析结果文件名为result.csv、survival_analysis_cn.png、survival_curve_cn.png

(2)

报告文件为report.md，导出后的文件为同名的pdf文件。

(3)

本小问中，我直接利用了前两问的原始数据建立了MySQL数据库，其中schema如下：

```
Database: diabetes_health
Table: health_indicators

Columns:
id (INT, primary key)
Diabetes_012 (DECIMAL(2,1))
HighBP (TINYINT)
HighChol (TINYINT)
CholCheck (TINYINT)
BMI (DECIMAL(5,1))
Smoker (TINYINT)
Stroke (TINYINT)
HeartDiseaseorAttack (TINYINT)
PhysActivity (TINYINT)
Fruits (TINYINT)
Veggies (TINYINT)
HvyAlcoholConsump (TINYINT)
AnyHealthcare (TINYINT)
NoDocbcCost (TINYINT)
GenHlth (TINYINT)
MentHlth (TINYINT)
PhysHlth (TINYINT)
DiffWalk (TINYINT)
Sex (TINYINT)
Age (TINYINT)
Education (TINYINT)
Income (TINYINT)
```

然后我通过与千问对话，尝试基于这个数据库让他通过写SQL代码来完成不同的任务，下面是我观察到的两个千问的失败案例：

Case1:

Q: 按 BMI 从高到低列出前 10 个 Diabetes_012=2 的样本

LLM WA:

```
SELECT
    id,
    Diabetes_012,
    BMI,
    Age,
    Sex,
    HighBP,
    HighChol,
    Smoker,
    PhysActivity,
    GenHlth
FROM health_indicators
WHERE Diabetes_012 = 2
ORDER BY BMI DESC
LIMIT 10;
```

My answer:

```
SELECT *
FROM health_indicators
WHERE Diabetes_012 = 2
ORDER BY BMI DESC
LIMIT 10;
```

Case2:

Q: 糖尿病患者的平均BMI是多少?

LLM WA:

```
SELECT
    ROUND(AVG(BMI), 2) AS Avg_BMI_Diabetic,
    COUNT(*) AS Sample_Size,
    MIN(BMI) AS Min_BMI,
    MAX(BMI) AS Max_BMI,
    ROUND(STDDEV(BMI), 2) AS BMI_StdDev
FROM health_indicators
WHERE Diabetes_012 = 2
    AND BMI IS NOT NULL;
```

My SELECT

```
    ROUND(AVG(BMI), 2) AS Avg_BMI_Diabetic,
    COUNT(*) AS Sample_Size,
    MIN(BMI) AS Min_BMI,
    MAX(BMI) AS Max_BMI,
    ROUND(STDDEV(BMI), 2) AS BMI_StdDev
FROM health_indicators
WHERE Diabetes_012 IN (1, 2)
    AND BMI IS NOT NULL;
```

很遗憾的是现在LLM发展的确实很快，我经过多次试验，没有发现干问给出的代码无法运行或者返回错误结果的现象。但是上面这两个例子仍然体现了LLM的局限性。在Case1中，我预期的结果是返回这 10 个样本的完整记录信息，也就是表中的全部字段。但 LLM 实际生成的 SQL 只选择了部分字段，而没有返回样本的全部字段信息。虽然这条 SQL 可以正确运行，并且筛选和排序逻辑也是正确的，但它并没有完整满足查询任务。我认为这个失败案例说明 LLM 在 text-to-SQL 场景下，可能会根据自己的判断擅自简化输出内容，只选择它认为重要的列。这种错误不会导致 SQL 报错，但会导致结果与用户真实需求不一致。Case2也同理，这里的糖尿病患者的定义不够明确，既可以包括前期也可以不包括，但是LLM自动在这一歧义处帮用户做了选择。在大型的任务中，这种prompt不够严谨的现象是难免的，而LLM缺少纠错或者追问机制，只会按照自己的理解执行任务，这样虽然可以执行成功，但未必满足用户真实需求，这一现象又难以发现，容易产生不良后果。

(4)

我创建的网页为<https://kiritanih.github.io/>，打开页面后，点击链接可以查看第（2）问的报告内容